

TP4 — Déployer un lab automatisé avec Docker et simulateur réseau

Formation NetOps : Automatiser la gestion de son Infrastructure Réseau — Ambient IT

Description

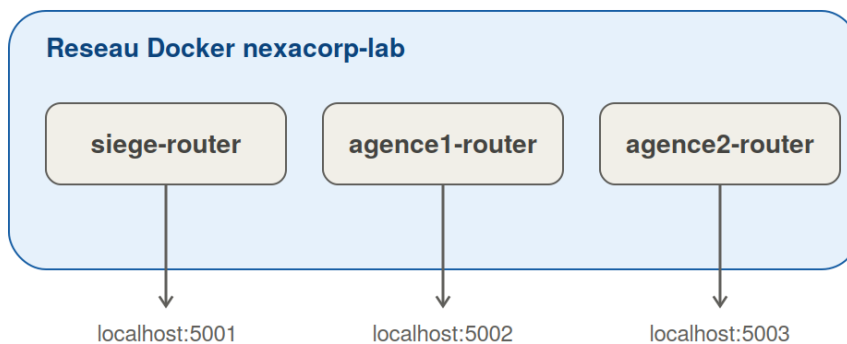


Figure 1: Lab Docker à plat (avant segmentation)

Ce TP transforme le mock d'API réseau du TP3 en un lab conteneurisé reproductible : au lieu d'un seul service Flask lancé à la main, trois conteneurs Docker représentent les trois sites de NexaCorp (Siège, Agence 1, Agence 2), démarrables en une seule commande (`docker compose up`). C'est la brique "lab" qui sera réutilisée pour tous les TP suivants (Ansible, ZDD, tests, etc.).

Prérequis

- TP1, TP2 et TP3 validés, et **la pull request du TP3 mergée dans dev** (le package `netops_client` doit être présent sur dev avant de démarrer ce TP — voir note à l'Étape 1)
- Un ordinateur avec au moins 4 Go de RAM disponible pour Docker
- Windows 10/11 avec virtualisation activée dans le BIOS (la plupart des PC récents l'ont par défaut)

Étapes

Étape 1 — Créer sa branche pour ce TP

```
cd netops-project-<nom-groupe>
git checkout dev
git pull origin dev
git checkout -b <prenom>-<nom>-jour1-tp4
```

Point d'attention : ce TP réutilise le package `netops_client` créé au TP3. Si la pull request du TP3 n'a pas encore été mergée dans dev, votre branche TP4 ne contiendra pas ce dossier et vous obtiendrez `ModuleNotFoundError: No module named 'netops_client'` à l'étape 9. Deux solutions :
- **Mergez d'abord la PR du TP3 dans dev** sur GitHub, puis `git merge dev` sur votre branche TP4 (solution recommandée, respecte le workflow) -
Ou récupérez le dossier directement depuis votre branche TP3 sans merger tout de suite : `bash git checkout <prenom>-<nom>-jour1-tp3 -- scripts/netops_client`

Étape 2 — Installer Docker Desktop

1. Télécharger Docker Desktop : <https://www.docker.com/products/docker-desktop/>
2. Lancer l'installateur, laisser l'option **"Use WSL 2 instead of Hyper-V"** cochée (recommandé)
3. Redémarrer le PC si demandé
4. Lancer Docker Desktop, attendre que l'icône de la baleine indique "Docker Desktop is running"

Vérifier dans Git Bash :

```
docker --version
docker compose version
```

Si docker n'est pas reconnu dans Git Bash après installation, fermez et rouvrez complètement le terminal. Si le problème persiste, vérifiez que Docker Desktop est bien lancé (icône dans la barre des tâches Windows).

Étape 3 — Rendre le mock d'API paramétrable par site

On modifie le mock du TP3 pour qu'il puisse représenter n'importe quel site selon une variable d'environnement, au lieu d'être toujours "router-siege-01" en dur.

```
cat > scripts/mock_api.py << 'EOF'
import os
from flask import Flask, jsonify

app = Flask(__name__)

SITE_NAME = os.environ.get("SITE_NAME", "site-inconnu")
```

```

INTERFACES = {
    "GigabitEthernet0/1": {"status": "up", "ip": "10.0.1.1", "vlan": 10},
    "GigabitEthernet0/2": {"status": "down", "ip": None, "vlan": 20},
    "GigabitEthernet0/3": {"status": "up", "ip": "10.0.3.1", "vlan": 30},
}

@app.route("/api/devices/<device_id>/interfaces", methods=["GET"])
def get_interfaces(device_id):
    return jsonify({"site": SITE_NAME, "device_id": device_id, "interfaces": INTERFACES})

@app.route("/health", methods=["GET"])
def health():
    return jsonify({"site": SITE_NAME, "status": "ok"})

if __name__ == "__main__":
    app.run(host="0.0.0.0", port=5000, debug=False)
EOF

```

Notez `host="0.0.0.0"` (au lieu de `127.0.0.1`) : indispensable pour qu'un conteneur Docker accepte les connexions venant de l'extérieur du conteneur.

Étape 4 — Écrire le Dockerfile

```

cat > lab/Dockerfile << 'EOF'
FROM python:3.12-slim

WORKDIR /app

RUN pip install --no-cache-dir flask

COPY scripts/mock_api.py .

EXPOSE 5000

CMD ["python", "mock_api.py"]
EOF

```

Étape 5 — Écrire le docker-compose : 3 sites NexaCorp

```

cat > lab/docker-compose.yml << 'EOF'
services:
  siege-router:
    build:
      context: ..
      dockerfile: lab/Dockerfile
    container_name: siege-router
    environment:
      - SITE_NAME=siege

```

```

    ports:
      - "5001:5000"
    networks:
      - nexacorp-lab

  agence1-router:
    build:
      context: ..
      dockerfile: lab/Dockerfile
    container_name: agence1-router
    environment:
      - SITE_NAME=agence1
    ports:
      - "5002:5000"
    networks:
      - nexacorp-lab

  agence2-router:
    build:
      context: ..
      dockerfile: lab/Dockerfile
    container_name: agence2-router
    environment:
      - SITE_NAME=agence2
    ports:
      - "5003:5000"
    networks:
      - nexacorp-lab

networks:
  nexacorp-lab:
    driver: bridge
EOF

```

context: .. : le build part de la racine du projet (pour pouvoir copier scripts/mock_api.py), tandis que dockerfile: lab/Dockerfile précise où trouver la recette. Chaque site est isolé dans son propre conteneur, mais tous connectés au même réseau virtuel nexacorp-lab — comme le hub-and-spoke vu en introduction.

Étape 6 — Démarrer le lab

```

cd lab
docker compose up -d --build

```

Vérifier que les 3 conteneurs tournent :

```

docker compose ps

```

Étape 7 — Tester chaque site

```
curl http://localhost:5001/health
curl http://localhost:5002/health
curl http://localhost:5003/health
```

Chaque réponse doit indiquer un site différent (siege, agence1, agence2).

```
curl http://localhost:5001/api/devices/router-siege-01/interfaces
```

Étape 8 — Explorer avec des commandes Docker/Linux de base

```
docker compose logs siege-router          # voir les logs d'un conteneur
docker exec -it siege-router sh           # ouvrir un shell dans le conteneur
  cat mock_api.py                         # (dans le conteneur) verifier le fichier
  exit                                    # sortir du conteneur
docker network inspect lab_nexacorp-lab    # voir les conteneurs connectes au reseau
```

Étape 9 — Adapter le client Python pour interroger les 3 sites

```
cat > scripts/check_all_sites.py << 'EOF'
from netops_client import NetworkClient, NetworkAPIError

SITES = {
    "siege": "http://localhost:5001",
    "agence1": "http://localhost:5002",
    "agence2": "http://localhost:5003",
}

if __name__ == "__main__":
    for site, url in SITES.items():
        client = NetworkClient(base_url=url)
        try:
            down = client.count_interfaces_down(f"router-{site}-01")
            statut = "OK" if not down else f"ATTENTION : {down}"
            print(f"{site:10s} -> {statut}")
        except NetworkAPIError as e:
            print(f"{site:10s} -> INJOIGNABLE ({e})")
EOF

cd ..
python scripts/check_all_sites.py
```

Résultat attendu :

```
siege      -> ATTENTION : ['GigabitEthernet0/2']
agence1    -> ATTENTION : ['GigabitEthernet0/2']
agence2    -> ATTENTION : ['GigabitEthernet0/2']
```

(Chaque site remonte la même interface en panne, car les 3 conteneurs exécutent le même mock répliqué.)

Étape 10 — Arrêter proprement le lab

```
cd lab
docker compose down
```

Étape 11 — Committer et pousser

```
cd ..
git add lab/ scripts/
git commit -m "TP4 : lab Docker reproductible avec 3 sites NexaCorp"
git push -u origin <prenom>-<nom>-jour1-tp4
```

Ouvrir une pull request <prenom>-<nom>-jour1-tp4 → dev.

Validation

- ☐ docker --version et docker compose version fonctionnent
- ☐ docker compose up -d --build (depuis lab/) démarre 3 conteneurs sans erreur
- ☐ docker compose ps montre les 3 conteneurs en état running
- ☐ Les 3 endpoints /health répondent chacun avec le bon nom de site
- ☐ python scripts/check_all_sites.py interroge les 3 sites et affiche un résultat par site
- ☐ docker compose down arrête proprement le lab
- ☐ Une pull request <prenom>-<nom>-jour1-tp4 → dev est ouverte